شرح كود Task 1 خطوة بخطوة

Machine Learning — Data Preparation & Exploration

Smart Traffic Accident Prediction

عن الملف ده	
بيشرح إيه؟	كل خلية كود في `Task1_Data_Preparation_EDA.ipynb` ، بنفس ترتيبها في الملف بالظبط
الداتا	Smart Traffic Management Dataset (Kaggle): 2000 صف × 12 عمود
الـ Target	`accident_reported` (0 = مفيش حادثة، 1 = فيه حادثة)
نوع المشكلة	Binary Classification
طريقة الشغل	نفس طريقة وكود الـ TA في سلايدات Data Foundations

المربع الرمادي = الكود من الملف بالظبط · المربع الأخضر = النتيجة · المربع البرتقالي = حاجة مهمة للمناقشة

قبل ما تبدأ: إزاي تفتح الملف وتشغله

1. افتح **VS Code**، ومن فوق اختار **File** ← **Open Folder**، واختار فولدر `ML project`.
2. دوس على `Task1_Data_Preparation_EDA.ipynb`. ولو VS Code طلب يثبت **Python** أو **Jupyter extension**، دوس Install.
3. أول مرة بس: افتح **Terminal** ← **New Terminal** واكتب:

```
python -m pip install -r requirements.txt
```

4. فوق على اليمين في الملف، دوس **Select Kernel** واختار Python.
5. عشان تشغل خلية: اقف عليها ودوس **Shift + Enter**. وعشان تشغل الملف كله: دوس **Run All**.
6. عشان تتنقل بين الخطوات بسرعة: افتح **OUTLINE** من الشريط اللي على الشمال تحت، هتلاقي فيه كل العناوين.

الـ Notebook فيه نوعين من الخلايا: **خلايا كود** (Python، والنتيجة بتظهر تحتها)، و**خلايا كلام** (Markdown) فيها العناوين والـ Observations. الملف ده بيشرح خلايا الكود، والـ Observations المكتوبة في الـ notebook هي اللي بتلخص معنى كل نتيجة.

خريطة الملف

الخطوات	القسم
1 – 2	العنوان + Library
3 – 10	Data Understanding: Read، Sample، Shape، Info، Columns، Describe، Unique، (1 Balance
11 – 15	Data Cleaning: Missing، Duplicates، **timestamp**، Consistency، Categorical check (2
16 – 17	Outlier Detection (Normal check، IQR، Z-score، Isolation Forest/LOF، Domain) + Treatment
18 – 22	EDA: Univariate، Feature vs Target، Feature-to-Feature، Correlation، GroupBy (3
23 – 24	حفظ الداتا النضيفة + Summary

الخطوة 1 العنوان وتعريف المشروع

أول خلية في الملف مش كود، دي كلام بس. بتقول إحنا بنعمل إيه: عندنا داتا مرور فيها 2000 قراءة من 5 أماكن، وعايزين نتنبأ هل حصلت **حادثة** ولا لأ. يعني الـ **target** (الحاجة اللي الموديل هيتنبأ بيها) هو `accident_reported`، وقيمته 0 (مفيش حادثة) أو 1 (فيه حادثة). ده اسمه **Binary Classification**، لأن الإجابة اختيار من اتنين بس. وكل الأعمدة التانية اسمها **features**، يعني المعلومات اللي هنستخدمها عشان نتنبأ.

الخطوة 2 استدعاء المكتبات: Library

قبل أي حاجة لازم نستدعي (import) المكتبات اللي هنشتغل بيها. المكتبة دي كود جاهز حد كتبه قبلنا، وإحنا بنستخدمه. أول 4 سطور هما نفس اللي في سلايد الـ TA بالظبط.

Code cell #1

```
# EDA
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Statistics & outlier detection
from scipy import stats
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import IsolationForest
from sklearn.neighbors import LocalOutlierFactor

import warnings
warnings.filterwarnings('ignore')
pd.set_option('display.max_columns', None)
sns.set_style('whitegrid')
```

- `import numpy as np` : مكتبة الحسابات الرياضية، و `np` اسم مختصر ليها عشان نكتب أقل.
- `import pandas as pd` : **أهم مكتبة**، هي اللي بتتعامل مع الجداول. الجدول في pandas اسمه **DataFrame**.
- `import matplotlib.pyplot as plt` : مكتبة الرسم الأساسية.
- `import seaborn as sns` : مكتبة رسم مبنية على matplotlib، بس أسهل ورسوماتها أحلى.
- `from scipy import stats` : مكتبة إحصاء، هنستخدمها للـ z-score والاختبارات الإحصائية.
- `StandardScaler` : بيوحد مقاييس الأعمدة (هنشرحه في الخطوة 16.4).
- `IsolationForest` و `LocalOutlierFactor` : طريقتين لكشف الـ outliers، الاتنين من سلايدات الـ TA.
- `warnings.filterwarnings('ignore')` : بيخفي رسايل التحذير عشان الملف يبقى نضيف. التحذيرات دي مش أخطاء.
- `pd.set_option('display.max_columns', None)` : بيخلي pandas يعرض كل الأعمدة وما يخبيش حاجة بـ "...".
- `sns.set_style('whitegrid')` : بيخلي خلفية الرسومات بيضا وفيها خطوط شبكة.

1) Data Understanding: فهم الداتا

القسم ده كله هدفه إننا **نتفرج على الداتا ونفهمها قبل ما نغير فيها أي حاجة**. زي ما الدكتور بيكشف على المريض قبل ما يكتبله علاج.

الخطوة 3 Read Data: قراءة الداتا

Code cell #2

```
# Read Data
data = pd.read_csv(r'Dataset/smart_traffic_management_dataset.csv')
df = data.copy()
df.head()
```

- `pd.read_csv(r'Dataset/...csv')` : بيقرا ملف الـ CSV ويحوله لجدول (DataFrame). والـ `r` قبل المسار معناها "raw string"، عشان الـ `/` أو `\` ما تتفهمش غلط.
- `df = data.copy()` : بنعمل **نسخة** اسمها `df` ونشتغل عليها، عشان الأصل `data` يفضل سليم لو غلطنا.
- `df.head()` : بيعرض **أول 5 صفوف**.

النتيجة: الملف اتقرا صح بالفاصلة العادية. ومش محتاجين `delimiter=';'` زي داتا الـ cardio بتاعة الـ TA.

الخطوة 4 Random Sample of Data: عينة عشوائية

Code cell #3

```
# Random Data
df.sample(5, random_state=42)
```

- `df.sample(5, random_state=42)` : بيجيب **5 صفوف عشوائي** من أي حتة في الجدول، مش من الأول بس. والـ `random_state=42` معناها إن "العشوائية" تطلع نفس الصفوف كل مرة تشغله، عشان النتيجة تبقى ثابتة. الرقم 42 مجرد عادة.

ليه بنعمل كده؟ عشان `head()` بيورينا الأول بس، وممكن تكون المشاكل في نص الملف.

الخطوة 5 Size of Data: حجم الداتا

Code cell #4

```
# Size of Data
df.shape
```

- `df.shape` : بيرجع (عدد الصفوف، عدد الأعمدة).

النتيجة: (2000, 12)، يعني 2000 صف و 12 عمود. وده حجم صغير، فلازم ما نمسحش صفوف من غير سبب قوي.

الخطوة 6 Information of Data: معلومات عن الأعمدة

Code cell #5

```
# Information of Data
df.info()
```

- `df.info()` : بيعرض لكل عمود اسمه، وكام قيمة مش فاضية فيه (Non-Null Count)، ونوعه (Dtype)، وحجم الجدول في الميموري.

النتيجة:

- كل الأعمدة فيها **non-null 2000**، يعني مفيش خانات فاضية.
- `timestamp` نوعه **نص** مش تاريخ، ودي **مشكلة هنصلحها في الخطوة 13**.
- `weather_condition` و `signal_status` نص، يعني أعمدة **categorical**.
- `location_id` متخزن رقم، بس هو في الحقيقة **كود لمكان** مش كمية. يعني المكان رقم 4 مش "ضعف" المكان رقم 2.

الخطوة 7 Check Names Of columns: أسماء الأعمدة

Code cell #6

```
# Check Names Of columns
df.columns
```

- `df.columns` : بيعرض أسماء كل الأعمدة.

النتيجة: الأسماء كلها مكتوبة بنفس الشكل (snake_case: حروف صغيرة و `_`). بس فيه أسماء مش واضحة:

- `traffic_volume` : مش معروف بالظبط بيقيس إيه.
- `avg_vehicle_speed` : مفيش وحدة، فافترضنا إنها km/h.
- `location_id` : مجرد رقم من غير اسم.

الخطوة 8 Check Description: الإحصائيات الوصفية

Code cell #7

```
# Check Description
df.describe().T
```

- `df.describe()` : بيحسب لكل عمود رقمي: count (العدد)، mean (المتوسط)، std (الانحراف المعياري، يعني القيم بعيدة عن المتوسط قد إيه)، min، 25%، 50% (وده الـ median)، 75%، max.
- `.T` : بتعمل **Transpose**، يعني بتقلب الجدول فالأعمدة تبقى صفوف، عشان يبقى أسهل في القراية.

> **النتيجة:** السرعة من 20 لـ 80، والحرارة من 10 لـ 35، والرطوبة من 30 لـ 90. وكل القيم منطقية. ولاحظ إن الـ mean قريب جداً من الـ median في كل الأعمدة، ودي علامة إن التوزيع متماثل. وكمان الـ mean بتاع `accident_reported` = 0.054 ، يعني 5.4% بس فيها حوادث.

Code cell #8

```
# Description of categorical columns
df.describe(include='object').T
```

- `df.describe(include='object')` : نفس الفكرة، بس للأعمدة **النصية**. بيطلع: count، و unique (عدد القيم المختلفة)، و top (أكتر قيمة متكررة)، و freq (اتكررت كام مرة).

> **النتيجة:** أكتر طقس متكرر Windy (438 مرة)، وأكتر إشارة Yellow (708 مرة).

الخطوة 9 Number of Unique Columns: عدد القيم المختلفة

Code cell #9

```
# Number of Unique Columns
df.nunique()
```

- `df.nunique()` : بيعد كام قيمة **مختلفة** في كل عمود.

> **النتيجة:**

- `timestamp` فيه 2000 قيمة مختلفة، يعني كل صف ليه وقت مختلف. فهو شبه ID، وما ينفعش يدخل الموديل كده، وهنطلع منه الساعة بس.
- السرعة والحرارة والرطوبة فيهم 2000 قيمة مختلفة برضه، بس ده طبيعي لأنهم أرقام عشرية.
- المكان 5 قيم، والطقس 5، والإشارة 3، والحادثة 2. دول أعمدة categorical.

الخطوة 10 Check Balance Of Data: توازن الـ target

Code cell #10

```
# Check Balance Of Data
df['accident_reported'].value_counts()
```

- `df['accident_reported']` : بنختار عمود الحادثة بس.
- `.value_counts()` : بيعد كل قيمة اتكررت كام مرة.

النتيجة: 0 ← 1892 صف، و 1 ← 108 صف.

Code cell #11

```
# Check Balance Of Data
df['accident_reported'].value_counts(normalize=True) * 100
```

- `value_counts(normalize=True)` : نفس العد، بس **كنسبة** من 1 بدل العدد.
- `* 100` : بنضرب في 100 عشان تبقى نسبة مئوية.

النتيجة: 94.6% مفيش حادثة، و **5.4%** فيه حادثة.

Code cell #12

```
plt.figure(figsize=(6, 4))
ax = sns.countplot(x='accident_reported', data=df, palette=['#4C9BE8', '#E8574C'])
for p in ax.patches:
    ax.annotate(f'{int(p.get_height())}', (p.get_x() + p.get_width() / 2, p.get_height()),
                ha='center', va='bottom')
plt.title('Balance of target: accident_reported')
plt.xticks([0, 1], ['0 = No accident', '1 = Accident'])
plt.show()
```

- `plt.figure(figsize=(6, 4))` : بنعمل مساحة رسم مقاسها 6 عرض × 4 طول.
- `sns.countplot(x='accident_reported', data=df, palette=[...])` : بيرسم عمود لكل قيمة طوله على قد عددها، و `palette` هي الألوان.
- الـ `for p in ax.patches:` بيلف على كل عمود في الرسمة، و `ax.annotate(...)` بيكتب الرقم فوق العمود.
- `plt.title` : عنوان الرسمة، و `plt.xticks` : بنكتب اسم واضح تحت كل عمود بدل 0 و 1.
- `plt.show()` : بيعرض الرسمة.

مهم جداً للمناقشة: الداتا دي **Imbalanced** (مش متوازنة)، النسبة تقريباً 17.5 لـ 1. ولو عملنا موديل يقول "مفيش حادثة" على طول، هياخد **accuracy 94.6%** وهو مش بيعمل أي حاجة! عشان كده في الـ Tasks الجاية هنقيس بـ **precision و recall و F1-score** (من Lecture 2)، ونقسم الداتا **stratified**، يعني الـ train والـ test الاتنين يبقى فيهم نفس النسبة.

2) Data Cleaning: تنضيف الداتا

هنا بندور على المشاكل اللي في الداتا ونقرر نعمل فيها إيه: خانات فاضية، صفوف متكررة، أنواع بيانات غلط، وقيم شاذة (outliers). **وأهم حاجة إن كل قرار يكون له سبب.**

الخطوة 11 Check Null (Missing Values): الخانات الفاضية

Code cell #13

```
# Check Null (Missing Values)
df.isna().sum()
```

- `df.isna()` : بيحط **True** مكان كل خانة فاضية و **False** مكان الخانة اللي فيها قيمة.
- `.sum()` : بيعد الـ True في كل عمود، لأن True بتتحسب 1 و False بتتحسب 0.

> **النتيجة: 0 في كل الأعمدة**، يعني مش محتاجين نملا حاجة.

> **ولو كان فيه missing values، كنا هنعمل إيه؟** (من سلايد Comparison of Imputation Methods)

نوع العمود	الطريقة	ليه؟
أرقام متصلة (سرعة، حرارة...)	Median	مش بيتأثر بالـ outliers، والداتا مش normal
عدد (عربيات، لوريات...)	Median	نفس السبب
categorical (طقس، إشارة)	Mode (الأكثر تكراراً)	ما ينفعش ناخد متوسط لكلمات
الـ target نفسه	نمسح الصف	ما ينفعش نخمن الإجابة

والكود اللي كنا هنستخدمه: `df['col'].fillna(df['col'].median())` أو `SimpleImputer(strategy='median')` .

الخطوة 12 Detect and drop duplicate rows: الصفوف المتكررة

Code cell #14

```
# Check Duplicated Values
print(f' Duplicated = {df.duplicated().sum()} \n==================================')
df[df.duplicated()]
```

- `df.duplicated()` : بيحط True قدام كل صف **نسخة طبق الأصل** من صف قبله.
- `.sum()` : بيعدهم. وجوه `print(f'...')` الـ `f` معناها f-string، يعني نقدر نحط قيمة جوه النص بين `{ }` .
- `df[df.duplicated()]` : بيعرض الصفوف المتكررة نفسها. ولقيناه جدول فاضي (Empty DataFrame).

> **النتيجة: 0** = Duplicated. ده نفس كود سلايد الـ TA بالظبط.

Code cell #15

```
# Check duplicates ignoring timestamp (same reading recorded twice at different times)
print(f' Duplicated (without timestamp) = {df.drop(columns="timestamp").duplicated().sum()}')
```

- `df.drop(columns='timestamp')` : بنشيل عمود الوقت **مؤقتاً** (من غير ما نمسحه من df).
- بعدها `.duplicated().sum()` : بندور على صفوف متطابقة في كل حاجة **ما عدا الوقت**.

ليه؟ لأن الوقت مختلف في كل صف، فالفحص العادي عمره ما هيلاقي تكرار. بس ممكن تكون نفس القراءة اتسجلت مرتين في دقيقتين مختلفتين. **النتيجة: 0 برضه.**

Code cell #16

```
# Drop Duplicated
df = df.drop_duplicates()
print(f' Duplicated = {df.duplicated().sum()} \n==================================')
print('Shape after dropping duplicates:', df.shape)
```

- `df = df.drop_duplicates()` : بيمسح الصفوف المتكررة ويحفظ النتيجة في `df` تاني.

النتيجة: الـ shape فضل (2000, 12) لأن مفيش تكرار. عملناها كخطوة **احتياطي** زي الـ TA.

الخطوة 13 Fix timestamp Data Type: تصليح نوع الـ timestamp

Code cell #17

```
# Convert timestamp from text to datetime
df['timestamp'] = pd.to_datetime(df['timestamp'])

print('First reading :', df['timestamp'].min())
print('Last reading  :', df['timestamp'].max())
print('Time step     :', df['timestamp'].diff().mode()[0])
df['timestamp'].dtype
```

ده الجزء اللي كنت بتدور عليه: **تصليح نوع الـ timestamp**.

- `df['timestamp'] = pd.to_datetime(df['timestamp'])` : **أهم سطر**. بيحول العمود من **نص** زي "2024-01-01 00:00:00" لـ **تاريخ حقيقي** (datetime). وبعدها نقدر نطلع منه الساعة واليوم، ونطرح تاريخين من بعض.
- `df['timestamp'].min()` و `.max()` : أول وآخر قراءة.
- `df['timestamp'].diff()` : بيحسب الفرق بين كل صف واللي قبله، و `.mode()[0]` : أكتر فرق متكرر.
- `df['timestamp'].dtype` : بيورينا النوع الجديد عشان نتأكد إن التحويل نجح.

النتيجة: النوع بقى datetime ✔. الداتا من 2024-01-01 00:00 لـ 2024-01-02 09:19، يعني **حوالي 33 ساعة بس**، وفيه قراءة كل دقيقة.

Code cell #18

```
# Extract useful time features
df['hour'] = df['timestamp'].dt.hour
df['is_peak_hour'] = df['hour'].isin([7, 8, 9, 16, 17, 18, 19]).astype(int)

# Readable label for location_id (used only for plots / tables)
location_map = {1: 'Ring Road Cairo', 2: 'Corniche Alexandria', 3: 'Suez Road',
                4: 'October Bridge', 5: 'Tahrir Square'}
df['location_name'] = df['location_id'].map(location_map)

# Total of the three vehicle counts
df['total_vehicles'] = df['vehicle_count_cars'] + df['vehicle_count_trucks'] +
df['vehicle_count_bikes']

df[['timestamp', 'hour', 'is_peak_hour', 'location_id', 'location_name',
'total_vehicles']].head()
```

- `df['hour'] = df['timestamp'].dt.hour` : الـ `.dt` بتفتح خصائص التاريخ، و `.hour` بتطلع **الساعة** (من 0 لـ 23). واخترنا الساعة بس، لأن اليوم والشهر والسنة ثابتين تقريباً (يومين بس).
- {c("df['hour'].isin([7, 8, 9, 16, 17, 18, 19])")}: بيسأل: هل الساعة دي من ساعات الذروة؟ وبيرجع True أو False. و {c('.astype(int)')} بيحولها لـ 1 أو 0. الناتج عمود {c('is_peak_hour')}.
- {c('location_map = {1: ...}')}: **Dictionary**، يعني قاموس فيه كل رقم وقصاده اسمه.
- {c("df['location_id'].map(location_map)")}: بيبدل كل رقم بالاسم اللي قصاده في القاموس، والناتج عمود {c('location_name')}. وده للعرض في الرسومات بس.
- {c("df['total_vehicles'] = cars + trucks + bikes")}: عمود جديد فيه مجموع كل المركبات.
- {c("df[[...]].head()")}: القوسين المربعين المزدوجين {c('[[ ]]')} معناهم إننا بنختار **كذا عمود** نعرضهم.

الخطوة 14 Consistency Check: هل الأرقام متسقة مع بعض؟

Code cell #19

```
same = (df['total_vehicles'] == df['traffic_volume']).sum()
bigger = (df['total_vehicles'] > df['traffic_volume']).mean() * 100
print(f'Rows where cars+trucks+bikes == traffic_volume : {same}')
print(f'Rows where cars+trucks+bikes  > traffic_volume : {bigger:.1f} %')
print('Correlation:', round(df['total_vehicles'].corr(df['traffic_volume']), 3))
```

- `(df['total_vehicles'] == df['traffic_volume']).sum()` : بيعد كام صف المجموع فيه بيساوي الـ volume بالظبط.
- `(df['total_vehicles'] > df['traffic_volume']).mean() * 100` : نسبة الصفوف اللي المجموع فيها **أكبر** من الـ volume. والـ mean بتاع True/False بيدينا النسبة على طول.
- `.corr(...)` : الارتباط بينهم.

النتيجة: اتساووا في **صف واحد بس**، وفي 48% من الصفوف المجموع أكبر من الـ volume، والارتباط ≈ 0. يعني `traffic_volume` **مش** مجموع العربيات، وبيقيس حاجة تانية. **القرار:** سيبنا الاتنين زي ما هما، لأننا مش عارفين أنهي فيهم الصح، وكتبناها كـ **limitation** (عيب في الداتا).

الخطوة 15 Categorical Values Check: فحص الأعمدة النصية

Code cell #20

```
for col in ['location_name', 'weather_condition', 'signal_status']:
    print(df[col].value_counts(), '\n')
```

- `for col in [...]:` : بيلف على الـ 3 أعمدة النصية واحد واحد، ويطبع `value_counts()` لكل واحد. " f"و `'\n'` معناها سطر جديد.

ليه؟ عشان نتأكد إن مفيش كلمة مكتوبة بأكتر من شكل، زي "Rainy" و "rainy". **النتيجة:** كله سليم، والفئات متوزعة كويس.

الخطوة 16 Outlier Detection: كشف القيم الشاذة

الـ **Outlier** قيمة بعيدة جداً عن باقي القيم. مثلاً لو كل السرعات من 20 لـ 80، وفجأة لقينا سرعة 900، يبقى دي غالباً غلطة. والـ outliers ممكن تلخبط الموديل، عشان كده لازم ندور عليها.

Select Method: حسب الـ flowchart بتاع الـ TA: لو بنبص على عمود واحد في المرة (**1D**) نستخدم **IQR أو Z-score**. ولو بنبص على كذا عمود مع بعض (**Multi-D**) نستخدم **Isolation Forest أو LOF**. إحنا هنعمل الاتنين.

Code cell #21

```
num_cols = ['traffic_volume', 'avg_vehicle_speed', 'vehicle_count_cars',
'vehicle_count_trucks',
            'vehicle_count_bikes', 'temperature', 'humidity']
num_cols
```

- **List** : `num_cols = [...]` فيها أسماء الأعمدة الرقمية المتصلة اللي هنفحصها.

ليه ما حطيناش كل الأعمدة الرقمية زي الـ TA؟ عشان `location_id` كود لمكان، و `accident_reported` هو الـ target (0/1)، و `hour` عمود جديد. مفيش معنى إننا ندور على "outlier" في عمود قيمه مجرد كود.

الخطوة 16.1 Check Normal Distribution: هل الداتا normal؟

لازم نعرف شكل توزيع كل عمود قبل ما نختار الطريقة، لأن الـ Z-score مبني على إن التوزيع **normal**، يعني شكل الجرس: معظم القيم في النص، وقليل منها على الأطراف.

Code cell #22

```
# Check Normal Distrbution
plt.figure(figsize=(15, 10))
for i, feature in enumerate(num_cols):
    plt.subplot(3, 3, i + 1)
    sns.histplot(df[feature], kde=True, bins=30, color="skyblue")
    plt.title(f"Distribution of {feature}")

plt.tight_layout()
plt.show()
```

ده **نفس كود الـ TA بالظبط** (سلايد Check Normal Distribution):

- `plt.figure(figsize=(15, 10))` : مساحة رسم كبيرة.
- `for i, feature in enumerate(num_cols):` : الـ `enumerate` بيدينا مع كل عمود رقمه: `i` = 0, 1, 2 ... و `feature` = اسم العمود.
- `plt.subplot(3, 3, i + 1)` : بنقسم المساحة لشبكة **3 صفوف × 3 أعمدة**، ونرسم في المربع رقم `i+1` . و (+1) لأن الترقيم في subplot بيبدأ من 1 مش 0.
- **Histogram** : `sns.histplot(df[feature], kde=True, bins=30, color='skyblue')` بيقسم القيم لـ 30 خانة (bins) ويرسم عمود لكل خانة طوله عدد القيم فيها. و `kde=True` بيرسم الخط الناعم اللي فوق الأعمدة.
- `plt.tight_layout()` : بيظبط المسافات عشان الرسومات ما تدخلش في بعض.

النتيجة: كل الرسومات **مفلطحة** (uniform)، مش شكل الجرس.

Code cell #23

```
# Skewness and normality test (Shapiro-Wilk: p < 0.05 means NOT normal)
pd.DataFrame({
    'skewness': df[num_cols].skew().round(3),
    'shapiro_p_value': [stats.shapiro(df[c])[1] for c in num_cols]
})
```

- `df[num_cols].skew()` : الـ **Skewness** بيقيس التوزيع مايل لناحية قد إيه. لو قريب من 0 يبقى متماثل، ولو موجب يبقى فيه ذيل لليمين، ولو سالب ذيل للشمال.
- `stats.shapiro(df[c])[1]` : اختبار **Shapiro-Wilk** بيسأل: هل الداتا normal؟ والـ `[1]` بتاخد الـ p-value. **لو الـ p-value أقل من 0.05 يبقى الداتا مش normal.**
- `pd.DataFrame({...})` : بنحط النتايج في جدول.
- `[ ... for c in num_cols]` : اسمها **list comprehension**، وهي طريقة مختصرة لعمل for loop في سطر واحد.

النتيجة: الـ skew ≈ 0 والـ p-values صغيرة جداً، يعني **مش normal**. **عشان كده IQR هو الطريقة الأساسية**، لأن الـ Z-score شرطه إن الداتا تكون normal.

الخطوة 16.2 IQR Rule: طريقة الـ Interquartile Range

نرتب القيم من الصغير للكبير، وناخد منها **Q1** (الربع الأول: 25% من القيم أصغر منه) و**Q3** (الربع التالت: 75% من القيم أصغر منه). الفرق بينهم اسمه **IQR = Q3 − Q1**. أي قيمة أصغر من `Q1 − 1.5×IQR` أو أكبر من `Q3 + 1.5×IQR` تبقى outlier.

Code cell #24

```
# Check Outlier
plt.figure(figsize=(15, 10))
for i, feature in enumerate(num_cols):
    plt.subplot(4, 2, i + 1)
    sns.boxplot(x=df[feature])
    plt.title(f'Box Plot of {feature}')
plt.tight_layout()
plt.show()
```

نفس كود الـ TA (سلايد Check Outlier IQR):

- `plt.subplot(4, 2, i + 1)` : شبكة 4 صفوف × 2 أعمدة.
- **Box Plot** : `sns.boxplot(x=df[feature])` ، والصندوق بيبدأ من Q1 ويخلص عند Q3، والخط اللي في النص هو الـ median. والخطين اللي على الجنب (whiskers) لحد IQR×1.5، **وأي نقطة برة الـ whiskers تبقى outlier**.

النتيجة: مفيش ولا نقطة برة الـ whiskers في أي رسمة.

Code cell #25

```
# Count IQR outliers per feature
iqr_table = []
for col in num_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    n_out = ((df[col] < lower) | (df[col] > upper)).sum()
    iqr_table.append([col, round(lower, 2), round(upper, 2), df[col].min(), df[col].max(),
n_out])

pd.DataFrame(iqr_table, columns=['feature', 'lower_limit', 'upper_limit', 'min', 'max',
'n_outliers'])
```

- `Q1 = df[col].quantile(0.25)` : القيمة اللي 25% من الداتا أقل منها.
- `Q3 = df[col].quantile(0.75)` : القيمة اللي 75% من الداتا أقل منها.
- `IQR = Q3 - Q1` ، و `lower = Q1 - 1.5 * IQR` ، و `upper = Q3 + 1.5 * IQR` : الحدود.
- `((df[col] < lower) | (df[col] > upper)).sum()` : الـ `|` معناها **OR** (أو). يعني بنعد القيم الأصغر من الحد الأدنى **أو** الأكبر من الحد الأعلى.

- `iqr_table.append([...])` : بنضيف صف للجدول في كل لفة، وفي الآخر بنحوله لـ DataFrame.

> **النتيجة: 0 outliers في كل الأعمدة.** وأقل وأكبر قيمة حقيقية في كل عمود جوه الحدود. مثلاً السرعة من 20 لـ 80، والحدود من −9.8 لـ 110.

الخطوة 16.3 Z-Score

الـ Z-score بيقول القيمة بعيدة عن المتوسط بكام **انحراف معياري** (std). القانون: `z = (x − mean) / std`. لو |z| أكبر من 3 تبقى outlier، لأن في التوزيع الـ normal، 99.7% من القيم بتكون بين −3 و +3.

Code cell #26

```
plt.figure(figsize=(15, 10))

for i, feature in enumerate(num_cols):
    plt.subplot(4, 2, i + 1)

    # Calculate Z-score
    z_scores = stats.zscore(df[feature])

    # (Scatter plot Z-scores)
    sns.scatterplot(x=np.arange(len(z_scores)), y=z_scores, s=8)

    # (Outlier threshold)
    plt.axhline(y=3, color='red', linestyle='--')
    plt.axhline(y=-3, color='red', linestyle='--')

    plt.title(f'Z-score of {feature}')
    plt.xlabel("Index")
    plt.ylabel("Z-score")

plt.tight_layout()
plt.show()
```

نفس كود الـ TA (سلايد Z-Score):

- `z_scores = stats.zscore(df[feature])` : بيحسب الـ z لكل قيمة: (القيمة − المتوسط) ÷ الانحراف المعياري.
- `np.arange(len(z_scores))` : أرقام من 0 لـ 1999 (رقم كل صف)، وهي اللي على المحور الأفقي.
- `sns.scatterplot(x=..., y=z_scores, s=8)` : بيرسم نقطة لكل صف، ارتفاعها = الـ z بتاعه. و `s=8` حجم النقطة.
- `plt.axhline(y=3, color='red', linestyle='--')` : خط أفقي أحمر متقطع عند +3، والسطر اللي بعده نفس الكلام عند −3.

> **النتيجة:** كل النقط بين الخطين الحمر.

Code cell #27

```
z_counts = {col: (np.abs(stats.zscore(df[col])) > 3).sum() for col in num_cols}
z_max = {col: round(np.abs(stats.zscore(df[col])).max(), 2) for col in num_cols}
pd.DataFrame({'n_outliers_|z|>3': z_counts, 'max_|z|': z_max})
```

- `{col: ... for col in num_cols}` : اسمها **dictionary comprehension**، بتعمل قاموس فيه لكل عمود رقم.
- `np.abs(...)` : القيمة المطلقة، يعني بتشيل السالب، عشان −4 و +4 الاتنين يتحسبوا.
- `> 3).sum()` : بيعد كام قيمة الـ z بتاعها أكبر من 3. و `.max()` : أكبر z في العمود.

النتيجة: 0 في كل الأعمدة، وأكبر |z| حوالي 1.8 بس. وده طبيعي، لأن التوزيع المفلطح عمره ما بيطلع قيم بعيدة.

الخطوة 16.4 Isolation Forest و LOF: الفحص على كل الأعمدة مع بعض

ممكن يبقى الصف عادي لو بصينا على كل عمود لوحده، بس **مجموعة القيم مع بعض** غريبة. الطريقتين دول بيبصوا على الصف كله مرة واحدة.

- **Isolation Forest:** بيحاول يعزل كل نقطة لوحدها بتقسيمات عشوائية، والنقطة اللي بتتعزل بسرعة تبقى شاذة.
- **LOF (Local Outlier Factor):** بيقارن الزحمة حوالين النقطة بالزحمة حوالين جيرانها، ولو النقطة في حتة فاضية أكتر بكتير من جيرانها، تبقى شاذة.

Code cell #28

```
X_scaled = StandardScaler().fit_transform(df[num_cols])

# Isolation Forest: we must choose a contamination rate (expected % of outliers) -> 1 %
iso = IsolationForest(contamination=0.01, random_state=42)
df['iso_outlier'] = iso.fit_predict(X_scaled)          # -1 = outlier, 1 = normal

# Local Outlier Factor: compares each point's local density with its neighbours
lof = LocalOutlierFactor(n_neighbors=20)
df['lof_outlier'] = lof.fit_predict(X_scaled)          # -1 = outlier, 1 = normal

print('Isolation Forest outliers :', (df['iso_outlier'] == -1).sum())
print('LOF outliers              :', (df['lof_outlier'] == -1).sum())
```

- `StandardScaler().fit_transform(df[num_cols])` : بيحول كل عمود بحيث يبقى متوسطه 0 وانحرافه 1. **ليه؟** لأن العربيات بتوصل لـ 899 والموتوسيكلات لـ 49 بس، فلو ما وحدناش المقاييس، الأعمدة الكبيرة هتطغى على الصغيرة.
- `IsolationForest(contamination=0.01, random_state=42)` : الـ `contamination` هي النسبة اللي **إحنا** متوقعين إنها outliers، وهنا 1%.
- `.fit_predict(X_scaled)` : بيتعلم من الداتا ويحكم على كل صف: **−1 = outlier** و **1 = عادي**.
- `LocalOutlierFactor(n_neighbors=20)` : بيقارن كل نقطة بأقرب **20 جار** ليها.

- `(df['iso_outlier'] == -1).sum()` : بيعد كام صف اتقال عليه outlier.

> **النتيجة:** Isolation Forest طلّع **20**، و LOF طلّع **0**.

Code cell #29

```
# Are the Isolation Forest rows really abnormal? Compare their ranges with the full data
pd.concat([df[num_cols].agg(['min', 'max']).T.add_prefix('all_'),
           df.loc[df['iso_outlier'] == -1, num_cols].agg(['min',
'max']).T.add_prefix('flagged_')], axis=1)
```

- `df[num_cols].agg(['min', 'max']).T` : أقل وأكبر قيمة في كل عمود، للداتا كلها.
- `df.loc[df['iso_outlier'] == -1, num_cols]` : الـ `.loc` بيختار صفوف بشرط. هنا بنختار الـ 20 صف اللي اتعلم عليهم بس.
- `.add_prefix(...)` : بيضيف كلمة قبل اسم العمود (all_ أو flagged_) عشان نفرق بينهم.
- `pd.concat([...], axis=1)` : بيلزق الجدولين جنب بعض.

> **الفكرة المهمة:** Isolation Forest طلّع 20 **عشان إحنا قلناله** يطلع 1% (2000 × 0.01 = 20)، فهو لازم يختار 20 حتى لو مفيش outliers أصلاً. والجدول بيثبت إن قيم الـ 20 دول كلها جوه المدى الطبيعي، فهما مش أخطاء.

Code cell #30

```
# Remove helper columns
df = df.drop(columns=['iso_outlier', 'lof_outlier'])
```

- `df.drop(columns=['iso_outlier', 'lof_outlier'])` : بنمسح العمودين المساعدين اللي عملناهم، عشان ما يدخلوش في التحليل بعد كده.

الخطوة 16.5 Domain (Logical) Check: فحص المنطق

غير الإحصاء، لازم نتأكد بعقلنا إن مفيش قيم **مستحيلة**، زي سرعة بالسالب، أو رطوبة أكتر من 100%، أو عدد عربيات بالسالب.

Code cell #31

```
checks = {
    'speed outside 0-200 km/h': ((df['avg_vehicle_speed'] < 0) | (df['avg_vehicle_speed'] >
200)).sum(),
    'negative vehicle counts': (df[['vehicle_count_cars', 'vehicle_count_trucks',
'vehicle_count_bikes']] < 0).any(axis=1).sum(),
    'humidity outside 0-100 %': ((df['humidity'] < 0) | (df['humidity'] > 100)).sum(),
    'temperature outside -20..55 C': ((df['temperature'] < -20) | (df['temperature'] >
55)).sum(),
    'traffic_volume <= 0': (df['traffic_volume'] <= 0).sum(),
}
pd.Series(checks, name='invalid_rows')
```

- `checks = {...}` : قاموس، كل مفتاح فيه وصف للفحص، وقيمته عدد الصفوف اللي فشلت فيه. مثلاً:
- `(df['avg_vehicle_speed'] < 0) | (df['avg_vehicle_speed'] > 200)` : سرعة بالسالب أو أكتر من 200؟
- `.any(axis=1)` : بيسأل: هل **أي** عمود من التلاتة سالب في الصف ده؟ و `axis=1` معناها بنبص على كل صف بالعرض.
- `pd.Series(checks, name='invalid_rows')` : بيحول القاموس لجدول بعمود واحد.

> **النتيجة: 0** في كل الفحوصات، يعني مفيش قيم مستحيلة.

الخطوة 17 Outliers Treatment Techniques: هنعمل إيه في الـ outliers؟

الـ TA شرح 5 طرق، وفي الملف جدول بيقارن بينهم:

الطريقة	معناها	تنفع معانا؟
Remove	نمسح الصف	لأ، مفيش outliers، وكمان عندنا 108 حادثة بس ومش عايزين نخسر داتا
Transformation	log أو جذر أو Box-Cox عشان نقلل الذيل الطويل	لأ، الداتا مفيهاش ذيل أصلاً (skew ≈ 0)
Imputation	نبدل الـ outlier بالـ median	لأ، مفيش حاجة نبدلها
Ignoring	نسيب القيم زي ما هي	**أيوه، دي اللي اخترناها**
Capping	نقص القيم عند الحدود	لأ، مش هتغير أي قيمة

وبعد الجدول فيه خليتين كود بيثبتوا القرار ده.

Code cell #32

```
# Remove Outliers Use IQR (lecture method) -> proof that it removes nothing
df_no_outliers = df.copy()
for col in num_cols:
    Q1 = df_no_outliers[col].quantile(0.25)
    Q3 = df_no_outliers[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    df_no_outliers = df_no_outliers[(df_no_outliers[col] >= lower) & (df_no_outliers[col] <=
upper)]

print("Original data shape:", df.shape)
print("Data shape after removing outliers:", df_no_outliers.shape)
```

ده **نفس كود الـ TA** (سلايد Remove Outliers Use IQR)، بس هنا بنستخدمه كـ **دليل**:

- `df_no_outliers = df.copy()` : نسخة جديدة.

- جوه الـ loop: بنحسب الحدود لكل عمود، وبعدين `df_no_outliers[(... >= lower) & (... <= upper)]` بيخلي **بس** الصفوف اللي جوه الحدود. والـ `&` معناها **AND** (و).

> **النتيجة:** الـ shape قبل (2000, 16) وبعد (2000, 16). يعني **مفيش ولا صف اتمسح**. (والـ 16 عمود بدل 12 عشان ضفنا 4 أعمدة في الخطوة 13.)

Code cell #33

```
# Capping -> proof that it changes nothing
changed = 0
for col in num_cols:
    Q1, Q3 = df[col].quantile([0.25, 0.75])
    IQR = Q3 - Q1
    capped = df[col].clip(Q1 - 1.5 * IQR, Q3 + 1.5 * IQR)
    changed += (capped != df[col]).sum()
print('Values changed by capping:', changed)
```

- `df[col].quantile([0.25, 0.75])` : بيحسب Q1 و Q3 مرة واحدة، و `Q1, Q3 = ...` بيوزعهم على متغيرين.
- `df[col].clip(lower, upper)` : ده الـ **Capping**. أي قيمة أصغر من الحد الأدنى بتبقى الحد الأدنى، وأي قيمة أكبر من الأعلى بتبقى الأعلى.
- `(capped != df[col]).sum()` : بيعد كام قيمة اتغيرت.

> **النتيجة: 0 قيم اتغيرت**. وده بيثبت إن القرار الصح هو **Ignoring**.

> **ملاحظة للـ Task الجاي:** الأعمدة مقاييسها مختلفة (0–49 مقابل 20–899)، فهنحتاج **Scaling** قبل KNN لأنه بيحسب مسافات. بس ده اسمه Scaling، مش علاج outliers.

3) Exploratory Data Analysis (EDA)

الـ EDA معناها إننا **نستكشف الداتا**: بندور على أنماط وعلاقات بالرسومات والجداول. والـ Task sheet بيطلب 5 أنواع، والملف ماشي عليهم بالترتيب.

الخطوة 18 Univariate Analysis: كل عمود لوحده

Uni يعني واحد. هنا بندرس كل عمود لوحده: متوسطه، انتشاره، وشكل توزيعه.

Numerical features: الأعمدة الرقمية

Code cell #34

```
uni_cols = num_cols + ['total_vehicles']
summary = df[uni_cols].agg(['mean', 'median', 'std', 'min', 'max', 'skew']).T
summary['range'] = summary['max'] - summary['min']
summary.round(2)
```

- `uni_cols = num_cols + ['total_vehicles']` : بنضيف العمود الجديد للقائمة. والـ `+` بين قائمتين بيلزقهم في بعض.
- `.agg(['mean', 'median', 'std', 'min', 'max', 'skew'])` : بيحسب الـ 6 إحصائيات دول لكل عمود مرة واحدة.
- `summary['range'] = summary['max'] - summary['min']` : المدى، يعني الفرق بين أكبر وأصغر قيمة.
- `.round(2)` : بيقرب لرقمين عشريين.

> **النتيجة:** الـ mean ≈ الـ median في كل الأعمدة (مثلاً السرعة 50.0 و 50.4)، يعني التوزيع **متماثل**.

Code cell #35

```
plt.figure(figsize=(16, 12))
for i, feature in enumerate(uni_cols):
    plt.subplot(4, 4, 2 * i + 1)
    sns.histplot(df[feature], kde=True, bins=25, color='#4C9BE8')
    plt.axvline(df[feature].mean(), color='red', linestyle='--', label='mean')
    plt.axvline(df[feature].median(), color='green', linestyle=':', label='median')
    plt.title(feature)
    plt.legend(fontsize=7)
    plt.subplot(4, 4, 2 * i + 2)
    sns.boxplot(x=df[feature], color='#A7CBEF')
    plt.title(f'{feature} (box)')
plt.tight_layout()
plt.show()
```

- `plt.subplot(4, 4, 2 * i + 1)` و `2 * i + 2` : شبكة 4×4، وكل عمود بياخد **مربعين جنب بعض**: histogram في الأول و boxplot في التاني.
- `plt.axvline(df[feature].mean(), color='red', linestyle='--', label='mean')` : خط **رأسي** أحمر عند المتوسط، والسطر اللي بعده خط أخضر عند الـ median.
- `plt.legend(fontsize=7)` : مربع صغير بيوضح كل خط معناه إيه.

> **النتيجة:** الخطين الأحمر والأخضر فوق بعض تقريباً. وكل التوزيعات مفلطحة، يعني كل القيم احتمالها متساوي تقريباً. وده غريب على داتا مرور حقيقية، و**علامة إن الداتا synthetic** (متولدة بالكمبيوتر).

Categorical features: الأعمدة النصية

Code cell #36

```
cat_cols = ['location_name', 'weather_condition', 'signal_status', 'hour']
plt.figure(figsize=(16, 9))
for i, col in enumerate(cat_cols):
    plt.subplot(2, 2, i + 1)
    order = sorted(df[col].unique()) if col == 'hour' else df[col].value_counts().index
    sns.countplot(x=col, data=df, order=order, color='#4C9BE8')
    plt.title(f'Count of {col}')
    plt.xticks(rotation=30 if col != 'hour' else 0)
plt.tight_layout()
plt.show()
```

- `cat_cols = ['location_name', 'weather_condition', 'signal_status', 'hour']`: الأعمدة اللي فيها فئات.
- `order = sorted(...) if col == 'hour' else df[col].value_counts().index`: لو العمود هو الساعة، رتّب من 0 لـ 23. غير كده، رتّب من الأكتر للأقل.
- `sns.countplot(x=col, data=df, order=order)`: عمود لكل فئة طوله عدد الصفوف فيها.
- `plt.xticks(rotation=30)`: بيميل الكتابة اللي تحت 30 درجة عشان ما تدخلش في بعض.

> **النتيجة:** الفئات متوازنة تقريباً (كل مكان حوالي 400 صف). والساعات من 0 لـ 8 أكتر من الباقي لأنها اتسجلت في اليومين.

الخطوة 19 Feature vs Target: كل عمود مع الحادثة

هنا السؤال الأهم: **هل العمود ده بيفرق في الحادثة؟** لو قيمه في الصفوف اللي فيها حادثة مختلفة عن الصفوف اللي مفيهاش، يبقى العمود ده مفيد للموديل.

الأعمدة الرقمية مع الحادثة

Code cell #37

```
plt.figure(figsize=(16, 10))
for i, feature in enumerate(uni_cols):
    plt.subplot(2, 4, i + 1)
    sns.boxplot(x='accident_reported', y=feature, data=df, palette=['#4C9BE8', '#E8574C'])
    plt.title(f'{feature} vs accident')
plt.tight_layout()
plt.show()
```

- `sns.boxplot(x='accident_reported', y=feature, data=df)`: بيرسم **صندوقين** جنب بعض لكل عمود: واحد للصفوف اللي مفيهاش حادثة (0)، وواحد للي فيها حادثة (1).

إزاي نقرا الرسمة؟ لو الصندوقين في نفس المستوى، يبقى العمود مش بيفرق. ولو واحد أعلى بوضوح من التاني، يبقى العمود له علاقة بالحادثة. **النتيجة:** الصندوقين متطابقين تقريباً في كل الأعمدة.

Code cell #38

```
# Mean of each feature per class + Mann-Whitney U test (p < 0.05 = significant difference)
rows = []
for col in uni_cols + ['hour']:
    no_acc = df.loc[df['accident_reported'] == 0, col]
    acc = df.loc[df['accident_reported'] == 1, col]
    rows.append([col, round(no_acc.mean(), 2), round(acc.mean(), 2),
                 round(stats.mannwhitneyu(acc, no_acc).pvalue, 3)])
pd.DataFrame(rows, columns=['feature', 'mean_no_accident', 'mean_accident', 'p_value'])
```

- `no_acc = df.loc[df['accident_reported'] == 0, col]` : قيم العمود في الصفوف اللي مفيهاش حادثة، والسطر اللي بعده نفس الكلام للي فيها حادثة.
- `stats.mannwhitneyu(acc, no_acc).pvalue` : اختبار **Mann-Whitney U** بيسأل: هل قيم المجموعتين مختلفة فعلاً، ولا الفرق صدفة؟ واخترناه بدل t-test لأنه مش محتاج الداتا تكون normal.

قاعدة الـ p-value (احفظها): لو **p أقل من 0.05**، فيه علاقة حقيقية (significant). ولو **p أكبر من أو يساوي 0.05**، الفرق ممكن يكون صدفة.

النتيجة: كل الـ p-values أكبر من 0.05، يعني **مفيش عمود رقمي ليه علاقة واضحة بالحادثة**. وأقربهم الحرارة (p = 0.071): الحوادث حصلت في حرارة أعلى شوية (23.7 مقابل 22.4).

الأعمدة النصية مع الحادثة

Code cell #39

```
overall_rate = df['accident_reported'].mean() * 100
plt.figure(figsize=(16, 9))
for i, col in enumerate(['location_name', 'weather_condition', 'signal_status',
'is_peak_hour']):
    plt.subplot(2, 2, i + 1)
    rate = df.groupby(col)['accident_reported'].mean().mul(100).sort_values(ascending=False)
    sns.barplot(x=rate.index, y=rate.values, color='#E8574C')
    plt.axhline(overall_rate, color='black', linestyle='--', label=f'overall
{overall_rate:.1f}%')
    plt.ylabel('Accident rate (%)')
    plt.title(f'Accident rate by {col}')
    plt.xticks(rotation=20)
    plt.legend()
plt.tight_layout()
plt.show()
```

- `overall_rate = df['accident_reported'].mean() * 100` : نسبة الحوادث العامة = 5.4%.
- `df.groupby(col)['accident_reported'].mean().mul(100)` : نسبة الحوادث **لكل فئة**، و `.mul(100)` يعني × 100. والمتوسط بتاع عمود فيه 0 و 1 هو نفسه النسبة.

- `.sort_values(ascending=False)` : ترتيب من الأعلى للأقل.
- `sns.barplot(x=rate.index, y=rate.values)` : عمود لكل فئة. و `plt.axhline(overall_rate)` : خط أسود عند النسبة العامة، عشان نقارن بيه.

> **إزاي نقرا الرسمة؟** العمود اللي فوق الخط الأسود فيه حوادث أكتر من المتوسط، واللي تحته أقل.

Code cell #40

```
# Chi-square test of independence (p < 0.05 = category and accident are related)
rows = []
for col in ['location_id', 'weather_condition', 'signal_status', 'is_peak_hour']:
    table = pd.crosstab(df[col], df['accident_reported'])
    rows.append([col, round(stats.chi2_contingency(table)[1], 3)])
pd.DataFrame(rows, columns=['feature', 'chi2_p_value'])
```

- `pd.crosstab(df[col], df['accident_reported'])` : جدول بيعد كل مزيج: مثلاً كام Rainy مفيهوش حادثة، وكام Rainy فيه حادثة.
- `stats.chi2_contingency(table)[1]` : اختبار **Chi-Square** للأعمدة اللي فيها فئات. بيسأل: هل الفئة والحادثة مرتبطين؟ والـ `[1]` هي الـ p-value.

> **النتيجة:** `signal_status` **هو الوحيد الـ significant (p = 0.038)**. أما المكان (p = 0.29) والطقس (p = 0.63) وساعة الذروة (p = 1.0)، فمفيش علاقة.

Code cell #41

```
pd.crosstab(df['signal_status'], df['accident_reported'], normalize='index').mul(100).round(1)
```

- `pd.crosstab(..., normalize='index')` : نفس الجدول، بس كل صف بيتحول لنسب (المجموع = 1)، وبعدين × 100.

> **النتيجة:** الإشارة الخضرا 6.6% حوادث، والحمرا 6.1%، والصفرا **3.7% بس**. بس خد بالك، دي علاقة **ضعيفة**، والحوادث كلها 108 بس.

الخطوة 20 Feature-to-Feature: الأعمدة مع بعضها

هنا بنشوف الـ features ليها علاقة ببعض ولا لأ. ولو عمودين شبه بعض جداً، يبقى ما ينفعش نحطهم الاتنين في الموديل.

Code cell #42

```
fig, axes = plt.subplots(1, 3, figsize=(17, 5))
sns.scatterplot(x='traffic_volume', y='avg_vehicle_speed', data=df, hue='accident_reported',
                palette=['#4C9BE8', '#E8574C'], alpha=0.6, s=15, ax=axes[0])
axes[0].set_title('Volume vs speed (expected: more traffic = slower)')
sns.scatterplot(x='vehicle_count_cars', y='traffic_volume', data=df, alpha=0.5, s=12,
ax=axes[1])
axes[1].set_title('Cars vs traffic_volume')
sns.scatterplot(x='vehicle_count_cars', y='total_vehicles', data=df, alpha=0.5, s=12,
ax=axes[2])
axes[2].set_title('Cars vs total_vehicles (engineered)')
plt.tight_layout()
plt.show()
```

- `fig, axes = plt.subplots(1, 3, figsize=(17, 5))` : طريقة تانية للرسم: بنعمل 3 رسومات جنب بعض مرة واحدة، و `axes[0]` و `axes[1]` و `axes[2]` هما الـ 3 رسومات.
- `sns.scatterplot(x=..., y=..., hue='accident_reported', alpha=0.6, ax=axes[0])` : نقطة لكل صف. و `hue` بيلون النقط حسب الحادثة، و `alpha` بيخلي النقط شفافة شوية، و `ax=` بيحدد هنرسم في أنهي مربع.

النتيجة:

- الزحمة مع السرعة: **سحابة عشوائية**. في الحقيقة المفروض الزحمة تقلل السرعة، بس هنا مفيش علاقة.
- العربيات مع الـ volume: مفيش علاقة، وده بيأكد اللي لقيناه في الخطوة 14.
- العربيات مع `total_vehicles` : **خط مستقيم** تقريباً، لأن العربيات هي معظم المجموع. وده اسمه **Multicollinearity**، يعني ما نحطش الاتنين في الموديل.

Code cell #43

```
fig, axes = plt.subplots(1, 2, figsize=(15, 5))
sns.boxplot(x='weather_condition', y='avg_vehicle_speed', data=df, color='#A7CBEF', ax=axes[0])
axes[0].set_title('Speed by weather (expected: rain/fog = slower)')
sns.boxplot(x='signal_status', y='traffic_volume', data=df, color='#A7CBEF', ax=axes[1])
axes[1].set_title('Traffic volume by signal status')
plt.tight_layout()
plt.show()
```

- `sns.boxplot(x='weather_condition', y='avg_vehicle_speed', ...)` : صندوق للسرعة لكل نوع طقس، والتاني صندوق للـ volume لكل إشارة.

النتيجة: السرعة حوالي 50 km/h في كل أنواع الطقس، يعني المطر والضباب **مش** بيبطأوا العربيات. وده مش منطقي في الواقع، فهو علامة تانية إن الداتا synthetic.

الخطوة 21 Correlation Analysis: الارتباط

الـ **Correlation** رقم من −1 لـ +1 بيقيس العلاقة الخطية بين عمودين:

- **+1**: لما الأول يزيد، التاني يزيد معاه.

- **−1**: لما الأول يزيد، التاني يقل.
- **0**: مفيش علاقة خطية.

وكقاعدة: أقل من 0.3 ضعيف، ومن 0.3 لـ 0.7 متوسط، وأكتر من 0.7 قوي.

Code cell #44

```
# location_id is left out: it is a category code, so a Pearson correlation on it has no meaning
corr_cols = uni_cols + ['hour', 'is_peak_hour', 'accident_reported']
corr = df[corr_cols].corr()

plt.figure(figsize=(12, 9))
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm', center=0, vmin=-1, vmax=1,
            linewidths=0.5, annot_kws={'size': 8})
plt.title('Correlation matrix (Pearson)')
plt.show()
```

- `corr = df[corr_cols].corr()` : بيحسب الارتباط (Pearson) بين **كل عمودين**، والناتج جدول مربع.
- شلنا `location_id` لأنه كود مكان، والارتباط على كود ملوش معنى. وعلاقته بالحادثة اختبرناها بـ chi-square في الخطوة 19.
- `sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm', center=0, vmin=-1, vmax=1)` : بيرسم الجدول **ألوان**. و `annot=True` بيكتب الرقم جوه كل مربع، و `fmt='.2f'` يعني رقمين عشريين. و `cmap='coolwarm'` : أحمر للموجب، وأزرق للسالب، ورمادي عند الصفر.

النتيجة: كل المربعات رمادي تقريباً (≈ 0). والاستثناء الوحيد: العربيات مع `total_vehicles` = **0.99** ، و `hour` مع `is_peak_hour` = 0.23 . والاتنين طبيعيين، لأن إحنا اللي عاملين العمود التاني من الأول.

Code cell #45

```
# Correlation of every feature with the target, strongest first
corr['accident_reported'].drop('accident_reported').sort_values(key=abs,
ascending=False).round(3)
```

- `corr['accident_reported']` : بناخد عمود الحادثة بس من جدول الارتباط.
- `.drop('accident_reported')` : بنشيل ارتباطه بنفسه (اللي دايماً = 1).
- `.sort_values(key=abs, ascending=False)` : بيرتب حسب **القيمة المطلقة**، عشان −0.04 و +0.04 يتحسبوا نفس القوة.

النتيجة: أقوى ارتباط هو الحرارة **+0.041**، وده **ضعيف جداً**. يعني مفيش عمود لوحده بيتنبأ بالحادثة. **بس** الـ Pearson بيقيس العلاقات الخطية بس، والـ Decision Tree ممكن يلاقي علاقات مش خطية.

GroupBy / Statistical Summary الخطوة 22

الـ `groupby` بيقسم الداتا لمجموعات، زي كل مكان لوحده أو كل نوع طقس لوحده، وبيحسب لكل مجموعة إحصائيات زي العدد والمتوسط والنسبة.

Code cell #46

```
# By location
df.groupby('location_name').agg(
    readings=('accident_reported', 'size'),
    accidents=('accident_reported', 'sum'),
    accident_rate_pct=('accident_reported', lambda s: s.mean() * 100),
    avg_volume=('traffic_volume', 'mean'),
    median_volume=('traffic_volume', 'median'),
    avg_speed=('avg_vehicle_speed', 'mean'),
).round(1).sort_values('accident_rate_pct', ascending=False)
```

ده اسمه **Named Aggregation**، وشكله: `اسم_العمود_الجديد=('العمود', 'العملية')` :

- `readings=('accident_reported', 'size')` : عدد الصفوف في كل مكان.
- `accidents=('accident_reported', 'sum')` : مجموع الـ 1، يعني **عدد الحوادث**.
- `accident_rate_pct=('accident_reported', lambda s: s.mean() * 100)` : الـ `lambda` **دالة صغيرة من غير اسم**، بتاخد العمود `s` وتطلع النسبة المئوية.
- `avg_volume=('traffic_volume', 'mean')` و `median_volume=('traffic_volume', 'median')` : متوسط ووسيط الزحمة.
- `.round(1).sort_values(...)` : نقرب لرقم عشري واحد ونرتب.

النتيجة: Tahrir Square أعلى مكان (28 حادثة من 409 = 6.8%)، و Ring Road Cairo أقل مكان (15 من 415 = 3.6%).

Code cell #47

```
# By weather
df.groupby('weather_condition').agg(
    readings=('accident_reported', 'size'),
    accidents=('accident_reported', 'sum'),
    accident_rate_pct=('accident_reported', lambda s: s.mean() * 100),
    avg_speed=('avg_vehicle_speed', 'mean'),
    avg_temperature=('temperature', 'mean'),
    avg_humidity=('humidity', 'mean'),
).round(1).sort_values('accident_rate_pct', ascending=False)
```

نفس الفكرة بالظبط، بس التقسيم حسب **الطقس**، وضفنا متوسط الحرارة والرطوبة.

النتيجة: Rainy أعلى (6.7%)، و Foggy أقل (4.3%). والسرعة ≈ 50 في كل الأنواع.

Code cell #48

```
# By signal status
df.groupby('signal_status').agg(
    readings=('accident_reported', 'size'),
    accidents=('accident_reported', 'sum'),
    accident_rate_pct=('accident_reported', lambda s: s.mean() * 100),
    avg_volume=('traffic_volume', 'mean'),
    avg_speed=('avg_vehicle_speed', 'mean'),
).round(1).sort_values('accident_rate_pct', ascending=False)
```

نفس الفكرة، بس حسب **الإشارة**.

> **النتيجة:** Yellow 3.7% > Red 6.1% > Green 6.6%. وده نفس اللي طلع في الـ chi-square.

Code cell #49

```
# Location x Weather accident rate (%)
pivot = df.pivot_table(index='location_name', columns='weather_condition',
                       values='accident_reported', aggfunc='mean').mul(100)
plt.figure(figsize=(9, 5))
sns.heatmap(pivot, annot=True, fmt='.1f', cmap='Reds')
plt.title('Accident rate (%) by location and weather')
plt.show()
```

- `df.pivot_table(index='location_name', columns='weather_condition', values='accident_reported', aggfunc='mean')` : **Pivot Table** زي اللي في Excel. الصفوف أماكن، والأعمدة أنواع طقس، وجوه كل خانة نسبة الحوادث للمكان ده في الطقس ده.
- `.mul(100)` : × 100. و `sns.heatmap(..., cmap='Reds')` : كل ما الخانة تبقى أحمر أغمق، كل ما نسبة الحوادث تكون أعلى.

> **خد بالك:** كل خانة فيها حوالي 80 صف بس، وحادثتين أو تلاتة. يعني الأرقام دي **مش مستقرة**، وما نبنيش عليها استنتاجات كبيرة.

Code cell #50

```
# By hour of day
hourly = df.groupby('hour').agg(accident_rate=('accident_reported', 'mean'),
                                avg_volume=('traffic_volume', 'mean'))
fig, ax1 = plt.subplots(figsize=(13, 4.5))
ax1.bar(hourly.index, hourly['accident_rate'] * 100, color='#E8574C', alpha=0.7,
label='accident rate (%)')
ax1.set_xlabel('Hour of day'); ax1.set_ylabel('Accident rate (%)')
ax2 = ax1.twinx()
ax2.plot(hourly.index, hourly['avg_volume'], color='#1a3a5c', marker='o', label='avg
traffic_volume')
ax2.set_ylabel('Average traffic volume')
ax2.grid(False)
ax1.set_xticks(range(24))
plt.title('Accident rate and traffic volume by hour', pad=28)
fig.legend(loc='upper center', ncol=2, bbox_to_anchor=(0.5, 0.94), frameon=False)
plt.show()
```

- `df.groupby('hour').agg(accident_rate=..., avg_volume=...)` : لكل ساعة: نسبة الحوادث ومتوسط الزحمة.
- `fig, ax1 = plt.subplots(...)` و `ax1.bar(...)` : أعمدة حمرا لنسبة الحوادث على المحور الشمال.
- `ax2 = ax1.twinx()` : بيعمل **محور رأسي تاني على اليمين**، عشان نرسم حاجتين ليهم مقاييس مختلفة على نفس الرسمة.
- `ax2.plot(..., marker='o')` : خط بنقط لمتوسط الزحمة. و `ax2.grid(False)` بيشيل الشبكة التانية عشان ما تتلخبطش.
- `fig.legend(...)` : مربع الشرح فوق الرسمة.

> **النتيجة:** نسبة الحوادث بتطلع وتنزل من غير نمط، من 0 حوادث الساعة 2 الضهر لـ 10 حوادث الساعة 8 بالليل. ومفيش ذروة الصبح والمسا.

الخطوة 23 Save the cleaned dataset: حفظ الداتا النضيفة

Code cell #51

```
clean_cols = ['timestamp', 'hour', 'is_peak_hour', 'location_id', 'location_name',
              'traffic_volume', 'avg_vehicle_speed', 'vehicle_count_cars',
'vehicle_count_trucks',
              'vehicle_count_bikes', 'total_vehicles', 'weather_condition', 'temperature',
              'humidity', 'signal_status', 'accident_reported']
df[clean_cols].to_csv('Dataset/traffic_cleaned_task1.csv', index=False)
print('Saved:', df[clean_cols].shape)
```

- `clean_cols = [...]` : الأعمدة اللي هنحفظها، يعني الـ 12 الأصليين + الـ 4 الجداد.
- `df[clean_cols].to_csv('Dataset/traffic_cleaned_task1.csv', index=False)` : بيحفظ الجدول في ملف CSV جديد. و `index=False` معناها ما يحفظش عمود أرقام الصفوف (0, 1, 2...).

> **النتيجة:** ملف فيه 2000 صف × 16 عمود، جاهز للـ Task 2 (الموديلات).

الخطوة 24 Summary of Main Findings: الملخص

آخر خلية كلام بس (مش كود)، وفيها ملخص كل اللي اكتشفناه، وجدول بكل قرارات التنضيف، وإيه اللي محتاجين نعمله في الـ Tasks الجاية. **دي أهم خلية تذاكرها قبل المناقشة.**

كلمات لازم تعرفها

الكلمة	معناها ببساطة
DataFrame	جدول في pandas (صفوف وأعمدة)
Feature	عمود بنستخدمه عشان نتنبأ (input / X)
Target / Label	اللي عايزين نتنبأ بيه (output / y)، وهنا `accident_reported`
Binary Classification	تصنيف لاختيارين بس (0 أو 1)
Imbalanced Data	فئة أكتر بكتير من التانية (94.6% مقابل 5.4%)
Missing Value / Null	خانة فاضية
Imputation	ملء الخانات الفاضية بقيمة (mean / median / mode)
Duplicate	صف متكرر طبق الأصل
Outlier	قيمة بعيدة جداً عن باقي القيم
IQR	Q3 − Q1، ومنه بنحدد حدود الـ outliers
Z-score	القيمة بعيدة عن المتوسط بكام انحراف معياري
Normal Distribution	توزيع شكله جرس (معظم القيم في النص)
Uniform Distribution	توزيع مفلطح (كل القيم احتمالها متساوي)
Skewness	ميل التوزيع لناحية (0 = متماثل)
p-value	لو أقل من 0.05: النتيجة حقيقية مش صدفة
Correlation	قوة واتجاه العلاقة بين عمودين (من −1 لـ +1)
Multicollinearity	عمودين شبه بعض جداً، فما نحطش الاتنين في الموديل
Scaling	توحيد مقاييس الأعمدة (مهم لـ KNN)
Synthetic Data	داتا متولدة بالكمبيوتر، مش من الواقع

أسئلة متوقعة في المناقشة

السؤال	الإجابة
الـ target بتاعك إيه؟ وليه؟	`accident_reported` . عمود حقيقي في الداتا، والإجابة 0 أو 1، فده Binary Classification زي مثال الـ cardio بتاع الـ TA.
الداتا balanced؟	لأ: 94.6% مقابل 5.4%. فالـ accuracy مضللة، وهنستخدم precision و recall و F1 وstratified split.
كان فيه missing values؟	لأ، 0. ولو كان فيه كنا هنستخدم median للأرقام و mode للـ categories.
ليه حولت الـ timestamp؟	لأنه كان متخزن نص. حولناه لـ datetime عشان نطلع منه الساعة.
ليه IQR مش Z-score؟	لأن الداتا مش normal (Shapiro p أقل من 0.05)، والـ Z-score شرطه إن الداتا تكون normal.
لقيت outliers؟ عملت إيه؟	لأ. IQR و Z-score و LOF و Domain check كلهم طلعوا 0، فاخترنا Ignoring. وكود الـ removal والـ capping ما غيروش حاجة.
ليه Isolation Forest طلّع 20؟	لأن contamination = 0.01، يعني إحنا قلناله يطلع 1% (2000 × 0.01 = 20). والقيم دي كلها طبيعية.
إيه أهم feature؟	`signal_status` ، وهو الوحيد اللي p = 0.038 (أقل من 0.05). الإشارة الصفرا 3.7% حوادث، والخضرا 6.6%.
ليه الـ correlation كله قريب من 0؟	الأعمدة مستقلة عن بعض، والداتا شكلها synthetic. فالموديلات في الـ Tasks الجاية غالباً هتجيب نتايج متواضعة.
ليه ما استخدمتش total_vehicles والعربيات مع بعض؟	الارتباط بينهم 0.99 (Multicollinearity)، فهنختار واحد بس.
traffic_volume مش بيساوي مجموع العربيات؟	صح، اتساووا في صف واحد بس. فهما بيقيسوا حاجتين مختلفين، وسيبنا الاتنين وكتبناها كـ limitation.
هتعمل إيه في الـ Task الجاي؟	One-hot encoding للأعمدة النصية، و Scaling للأرقام، ومعالجة الـ imbalance، وبعدين KNN و Decision Tree و Naive Bayes.